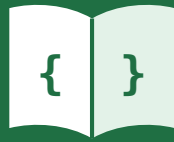

Modül 13 — Reactive Programlama ve WebFlux

Ödevler

Spring Boot Edu



2026-09-24

İçindekiler

Nasıl Çalışılır	2
Ödev 1 — Reactive Kitap API'si (Kolay)	2
Ödev 2 — SSE ile Canlı Stok Akışı (Orta)	3
Ödev 3 — Üç Kaynaktan Bir Ürün Sayfası (Zor)	3
Ek Meydan Okuma (İsteğe Bağlı)	4

Nasıl Çalışılır

1. Başlangıç kodu `modules/13-reactive/exercise/` altındadır. TODO yorumlarını bulun.
2. Ödevler Docker istemez: Verilen servisler verilerini bellekte tutar ama tamamen reactive'dir.
3. Testleri çalıştırın:

```
./mvnw -Pexercises -pl modules/13-reactive/exercise -am test
```

4. Tüm testler yeşil olduğunda ödev tamamdır.
5. Takılırsanız önce ipuçlarını, sonra `modules/13-reactive/solution/` altındaki çözümü okuyun.

Ödev 1 — Reactive Kitap API'si (Kolay)

Hedef: Verilen BookStore'daki kitapları **functional endpoint'lerle** bir REST API olarak sunmak.

İstek	Yanıt
GET /api/books	200, tüm kitaplar
GET /api/books/{isbn}	kitapla 200 veya 404
POST /api/books	Location: /api/books/{isbn} ile 201
DELETE /api/books/{isbn}	204, kitap yoksa 404

Yapılacaklar (`exercisel.BookRoutes`):

- TODO 1a — /api/books önekli bir `RouterFunction<ServerResponse>` bean'i.
- TODO 1b — İki GET yolu.
- TODO 1c — POST.
- TODO 1d — DELETE.

İpuçları:

- Ders bölüm 3.4: `route().path("/api/books", api -> api.GET(...).POST(...)).build()`.
- Bilinmeyen bir ISBN için `books.find(isbn)` boştur: `switchIfEmpty(ServerResponse.notFound()).build()`.
- `books.delete(isbn)` true veya false yayar: `flatMap(deleted -> deleted ? ... : ...)` ile karar verin.
- Bean metoduna sınıftan farklı bir ad verin (ör. `bookRouter`), aksi hâlde iki bean adı çıkarır.

Kabul kriterleri: `Exercise1Test` içindeki 4 testin hepsi geçer.

Tahmini süre: 25 dakika

Ödev 2 — SSE ile Canlı Stok Akışı (Orta)

Hedef: Bir depo ekranı stok seviyelerini canlı gösterir. Her değişiklik tüm açık ekranlara gönderilir. Bir ekran tek bir kitabı takip edebilir.

StockFeed hazır verilmiştir: `publish(level)` bir değişiklik gönderir, `changes()` tüm değişikliklerin sıcak (hot) bir Flux'ıdır.

Yapılacaklar (`exercise2.StockController`):

- TODO 2a — Yeni miktar JSON gövdesiyle `PUT /api/stock/{isbn}`: bir `StockLevel` yayınlayın, 204 ile yanıtlayın.
- TODO 2b — `text/event-stream` olarak `GET /api/stock/stream`: her değişiklik bir Server-Sent Event olsun.
- TODO 2c — `?isbn=...` ile yalnızca o kitabın değişiklikleri.
- TODO 2d — Client yanıt header'larını hemen alsın diye önce bir yorum event'i gönderin.

İpuçları:

- Ders bölüm 3.6 deseni eksiksiz gösterir (`ServerSentEvent.builder(...)`, `startWith(...)`).
- `@RequestParam(required = false) @Nullable String isbn` ve `filter(...)`.
- `@RequestBody int quantity`, 7 gibi düz bir JSON sayısını okur.

Kabul kriterleri: `Exercise2Test` içindeki 3 testin hepsi geçer.

Tahmini süre: 30 dakika

Ödev 3 — Üç Kaynaktan Bir Ürün Sayfası (Zor)

Hedef: Ürün sayfası fiyatı, stoğu ve puanı gösterir. Her biri farklı bir servisten gelir ve her biri yaklaşık 300 ms sürer. Sayfa 900 ms değil 300 ms sürsün diye onları **paralel** yükleyin. Puan isteğe bağlıdır: Eksik, hatalı veya çok yavaş olduğunda da sayfa çalışmalıdır.

Yapılacaklar (`exercise3.ProductPage.load`):

- TODO 3a — Puan: puan yoksa, hata varsa veya 1 saniyeden uzun sürerse → 0.0.
- TODO 3b — Üç servise aynı anda sorun ve yanıtları bir `ProductView`'da birleştirin.
- TODO 3c — Fiyat yoksa (bilinmeyen kitap) sonuç boştur.

İpuçları:

- Ders bölüm 3.5: `Mono.zip(a, b, c).map(t -> ...t.getT1()...)`. Kaynaklardan biri boş olur olmaz zip de boştur. TODO 3c bu yüzden kendiliğinden çözülür.
- `timeout(Duration.ofSeconds(1), onErrorReturn(0.0), defaultIfEmpty(0.0))`. Sırayı düşünün: Zaman aşımı da bir hatadır.
- Testler sanal zaman kullanır: Kodunuz kaynakları arka arkaya yüklerse `expectNoEvent(299 ms)` başarısız olur.

Kabul kriterleri: Exercise3Test içindeki 5 testin hepsi geçer.

Tahmini süre: 40 dakika

Ek Meydan Okuma (İsteğe Bağlı)

Uygulamaya ProductPage sonucunu sunan bir GET `/api/products/{isbn}` ekleyin (boş sonuç için 404). Dersin `/api/books/{isbn}` endpoint'ini çağıran, WebClient kullanan bir fiyat servisi yazın. Ardından bir servis kapalıyken sayfanın ne kadar sürdüğünü `curl -w "%{time_total}"` ile ölçün.